

21017030 민선홍 웹프로그래밍 홈페이지 과제

1. 로그인 화면(login.html)



login.html을 실행하게 되면 다음과 같은 화면이 나오게 된다.

```
<style>
#logobox{
  height:60px;
  margin: 0 auto;
  text-align:center;
  background-color: #rgb(230, 230, 230);
  /* Permalink - use to edit and share this gradient: http
  background: linear-gradient(to right, #rgba(10,77,153,
}
.logo{
  width:200px;
  height:50px;
}
.loginPage{
  text-align: center;
  width:500px;
  height:300px;
  margin: 200px auto;
  padding: 20px;
  border:1px solid #ccc;
  border-radius: 20px;
  box-shadow: 5px 5px 10px #rgb(120, 120, 120), -5px -5px

```

- 중앙 정렬을 하기 위해서 margin: [값] auto로 구성을 했다.
 - 로그인 화면 뿐만 아니라 메인 화면과 대분류 카테고리의 화면 내에서도 [값]을 바꿔 높이를 조절하면서 많이 이용.
 - 가장 상단에 있는 로고가 있는 바탕색은 6주차 강의 자료에서 배웠던 'Ultimate CSS Gradient Generator'와 그레디언트 속성을 활용해서 그라데이션 색 적용.
 - box-shadow를 이용해서 로그인 페이지를 그림자 효과 적용
- 로그인 페이지 뿐만 아니라 메인 화면의 하단 버튼, 상단 바, 시작 페이지 제목에도 그림자를 많이 적용해서 효과를 많이 줌

```
<button onclick="login()">로그인</button>
</div>

<script>
function login(){
  let id = document.getElementById("userId").value;
  let pw = document.getElementById("userPw").value;

  const USER_ID = "민선홍";
  const USER_PW = "21017030";

  if(id === USER_ID && pw === USER_PW){
    alert("로그인 성공!");
    window.location.href = "/main/main.html"; //window.location.href를 사용하여 페이지 이동
  } else {
    alert("아이디 또는 비밀번호가 틀렸습니다.");
  }
}
</script>
</body>
</html>

```

로그인 기능 구현

- 로그인 버튼을 클릭 했을 때 등록한 이벤트 onclick으로 login()함수를 호출
- getElementById().value로 텍스트 박스에 입력된 값들을 불러와 내 학번과 비밀번호와 일치 여부 확인.
- 틀리면 alert()만 실행, 같으면 main.html로 이동함
- (이때 지금까지 많이 사용했던 앵커 태그(<a>)를 사용하지 않고 버튼을 쓰기 위해 새로운 형태인 **window.location.href**를 사용. 해당 값의 의미는 다음과 같다.
window.location 자체에는 브라우저의 주소창과 관련된 정보를 담고 있는 객체이고 .href를 하게 된다면 현재 페이지의 전체 URL 문자열 정보가 저장되어 있다. 따라서 'window.location.href=이동주소'를 하면 다른 페이지로 이동이 가능하다)

2. 메인 화면(main.html)



- 구성요소: 로고, 대분류 카테고리 탭, 우측 상단 시계, 콤보 박스를 이용한 대분류, 소분류 카테고리 이동, 4개의 이미지가 있는 슬라이드 형식 배경, 하단 아이콘이 있는 대분류 이동 버튼.

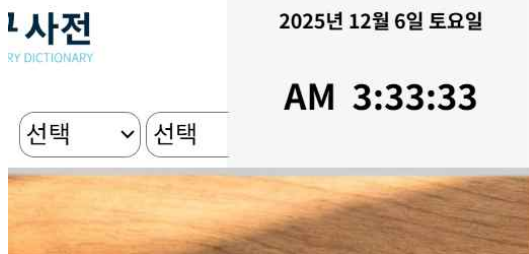
2.1. 상단 카테고리 탭



```
#time{
  position:fixed;
  right:0; top:0px;
  width:15%;
  height:120px;
  min-width: 200px;
  background-color: ■rgb(240, 240, 240);
  text-align: center;
  box-shadow: 0px 5px 2px ■rgb(200, 200, 200);
  z-index:102; /* 버튼의 스케일이 커질때 main
```

우측 상단 시계 부분(tabDesign\mainTab.css)

- 크기를 조절할 때 단순히 픽셀 단위를 주지 않고 퍼센트를 줘서 화면 크기에 따라 카테고리 배치 간격과 상단 바, 시계 바 길이가 변하게 됨.
- min-width는 최소 너비를 지정하는 것으로 이걸 지정하지 않으면 간격이 너무 좁아지거나 공간이 없어서 태그들이 아래로 내려가 버리는 현상을 막음.
- 시계는 position을 fixed를 하고 right, top을 0으로 줘서 우측 상단에 고정
- min-width로 제한을 줘서 다른 것을 가려서라도 시계가 보이게끔 함



2.2. 카테고리 이동 기능

```
const firstCategoryUrls = { // 대분류 카테고리 URL 매핑
  mechanical: 'category/mechanicalPencilCategory.html',
  ballpoint: 'category/ballpointPenCategory.html',
  fountain: 'category/fountainPenCategory.html',
  notebook: 'category/notebookCategory.html',
  eraser: 'category/eraserCategory.html'
};

const secondCategoryUrls = { // 소분류 카테고리 URL 매핑
  mechanical: [{text: '[자바펜] 메탈그립사프', url: './category/mechanicalPencil/metalgr',
    {text: '[자바펜] 자바 제도', url: './category/mechanicalPencil/javaajedo.
    {text: '[펜넬] 그래프1000', url: './category/mechanicalPencil/graph1000
    {text: '[펜넬] P20x', url: './category/mechanicalPencil/p205.html'},
    {text: '[펜넬] 스매쉬', url: './category/mechanicalPencil/smash.html'},
    {text: '[파이롯트] S20', url: './category/mechanicalPencil/s20.html'},
    {text: '[파이롯트] S30', url: './category/mechanicalPencil/s30.html'},
    {text: '[미쓰비시 연필] 알파걸 슬림', url: './category/mechanicalPencil/a
    {text: '[미쓰비시 연필] 알파걸 혼들사프', url: './category/mechanicalPencil
    {text: '[파카] 조더 오리지널', url: './category/mechanicalPencil/jotter.!
```

```
function firstCategorySelect(){ // 대분류 선택 시 소분류 에 정보를 넣음 및 수정
  const firstCategory = document.getElementById('first-category-select');
  const secondCategory = document.getElementById('second-category-select');

  const selected = firstCategory.value;
  secondCategory.innerHTML = '<option value="">선택</option>'; //기본 옵션 초기화

  // 대분류를 아직 안 골랐거나, 해당 소분류 리스트가 없으면 끝
  if (!selected || !secondCategoryUrls[selected]) return;

  const options = secondCategoryUrls[selected];
  let optionHtml = '';
  for (let i in options){
    optionHtml += '<option value=' + options[i].url + '>' + options[i].text + '</option>';
  }
  secondCategory.innerHTML += optionHtml;

function categoryMove(){ //이동 버튼 클릭
  const firstCategory = document.getElementById('first-category-select');
  const secondCategory = document.getElementById('second-category-select');

  const selected = firstCategory.value;
```

코드 구현 부분

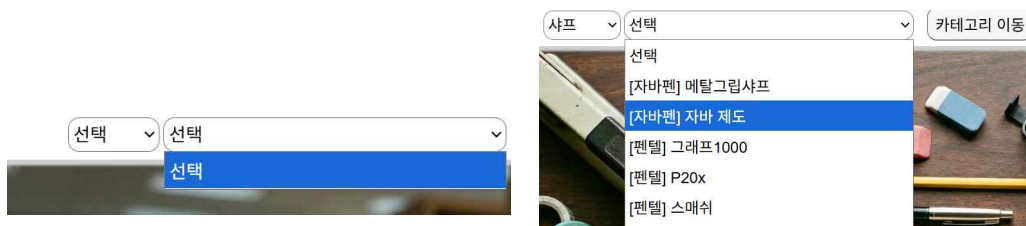
```
<div class = "second">
  <div class="search">
    <select id="first-category-select" onchange="firstCategorySelect()">
      <option value="">선택</option>
      <option value="mechanical">사프</option>
      <option value="ballpoint">볼펜</option>
      <option value="fountain">만년필</option>
      <option value="notebook">공책</option>
      <option value="eraser">지우개</option>
    </select>
    <select id="second-category-select">
      <option value="">선택</option>
    </select>

    <input type="button" id="categoryMove" value="카테고리 이동" onclick="categoryMove()">
  </div>
</div>
```

body 부분

- script 내에서 12주차 강의 자료에서 나왔던 객체를 이용(키를 이용해서 접근이 가능하다는 장점을 이용)

- 대분류 카테고리에는 option의 value값을 key로 이용해서 main.html 기준 url 이동 경로를 지정.
- 대분류 카테고리, 서브 카테고리 이동을 동시에 구현해야 하기 때문에 서브 카테고리 부분 option은 대분류가 선택 될 때 반복문으로 innerHTML로 옵션을 집어 넣음(결과적으로 대분류가 선택되기 전에는 서브 카테고리 콤보 박스가 보이지 않게 만들 수 있음).



선택전

선택후

- 자바 스크립트 구현은 <select> 태그에 입력 값이 변경될 때 발생하는 이벤트 onchange를 등록.
- 서브 카테고리의 객체는 대분류 카테고리의 value 값을 키로 이용해서 객체 배열 형태로 저장
- 서브 카테고리의 value에 서브 카테고리의 url를 저장해서 버튼 이벤트가 발생할 때 선택된 값을 바로 가져와 페이지 이동을 쉽게 함.
- 버튼을 누르면 각 카테고리의 선택 상태에 따라 이동을 함.
- url 이동은 똑같이 window.location.href를 이용.

```
function categoryMove(){ //이동 버튼 클릭
    const firstCategory = document.getElementById('first-category-select');
    const secondCategory = document.getElementById('second-category-select');

    const selected = firstCategory.value;
    const sub = secondCategory.value;
    if(selected == ""){
        alert("먼저 대분류부터 선택해주세요.");
        return;
    }
    if(sub == ""){
        window.location.href = firstCategoryUrls[selected];
    }
    if(sub != ""){
        window.location.href = secondCategory.value;
    }
}
```

2.3. 중앙 사진 슬라이드 및 자동 슬라이드 기능

```
// 슬라이더 구현 부분
$(document).ready(function(){
    let count = $('#div.content').length; // 총 이미지 개수
    let currentIndex = 0; // 현재 이미지 인덱스
    let clickedButton = false; //버튼 클릭 여부 확인 변수
    //페이지 이동(600ms 동안 left값을 변경)
    let changePicture = function(){
        $('#.content-container').animate({left: -1100 * currentIndex}, 500);
    }
    //이벤트 연결
    $('#slider-button-left').click(function(){
        if(currentIndex > 0){
            currentIndex = currentIndex - 1;
            clickedButton = true;
            changePicture();
        }
        //가장 첫번째 이미지에서 왼쪽 버튼 클릭 시 마지막 이미지로 이동
        else if(currentIndex == 0){
            currentIndex = count - 1;
            clickedButton = true;
            changePicture();
        }
    });
    $('#slider-button-right').click(function(){
        if(currentIndex < count - 1){
            currentIndex = currentIndex + 1;
            clickedButton = true; //버튼 클릭 여부 true로 변경
            changePicture();
        }
        //가장 마지막 이미지에서 오른쪽 버튼 클릭 시 첫번째 이미지로 이동
        else if(currentIndex == count - 1){
            currentIndex = 0;
            clickedButton = true; //버튼 클릭 여부 true로 변경
            changePicture();
        }
    });
});
```

버튼 이벤트 동작 코드

```
//자동 슬라이더
setInterval(function(){
    if(!clickedButton){ //버튼 클릭이 없을 때만 자동
        if(currentIndex < count - 1){
            currentIndex = currentIndex + 1;
            changePicture();
        }
        else if(currentIndex == count - 1){
            currentIndex = 0;
            changePicture();
        }
    }
    else{
        //버튼 클릭 후 10초 뒤에 자동 슬라이더 재작동
        setTimeout(function(){ // 특정 시간이 지났을 때
            clickedButton = false;
        }, 10000);
    }
}, 5000); //5초마다 슬라이더 변경
```

자동 슬라이드 코드

```
<!--슬라이더 구현-->
<div id="content-main">
    <input type="button" id="slider-button-left" value="←">
    <div id="content-slider">
        <div id="content-title"><p id="big-title">문구 사전</p>
        <p id="small-title">다양한 문구를 한 눈에 보기</p></div>
        <div class="content-container">
            <div class="content">
                
            </div>
            <div class="content">
                
            </div>
            <div class="content">
                
            </div>
            <div class="content">
                
            </div>
        </div>
    </div>
    <input type="button" id="slider-button-right" value="→">
</div>
```

HTML 부분

- 구현은 jquery를 이용, 버튼 이벤트 동작은 14주차 강의자료의 슬라이더 부분과 유사.
- 차이점: 버튼과 슬라이드 부분을 잘 구성시키기 위해 버튼까지 전부 담은 영역을 만듦



```
#content-main{
    width:1300px;
    height:600px;
    overflow:hidden;
    position:relative;
    text-align: center;
    margin: 120px auto 0 auto;
}

#content-slider{
    display:inline-block;
    width: 1100px;
    height: 600px;
    margin: 0 auto;
    position:relative;
    overflow:hidden;
}
```

- 추가 기능:

가장 마지막에 있는 이미지에서 오른쪽 버튼을 누른 경우->처음으로 돌아감
첫 이미지에서 왼쪽 버튼->마지막 이미지로 이동

- 자동 슬라이드: setInterval를 이용해서 일정 주기 동안 오른쪽으로 슬라이드 됨.
만약 버튼을 누른 경우 clickedButton 변수가 false가 되고 setTimeout을 실행하게 됨. setTimeout이란 해당 함수를 실행하기 전에 설정한 시간만큼 기다리는 것.
여기서는 10초 정도로 설정했으며 10초 동안에는 자동 슬라이드가 멈추게 된다.

```
}
#slider-button-left, #slider-button-right{
  position: absolute;
  top: 50%; /*요소 중앙 배치 9주차 활용*/
  margin-top: -100px;
  width: 50px;
  height: 200px;
  font-size: 40px;
  font-weight: bold;
  cursor: pointer;
  opacity: 0.6;
}
```

css 속성

- 버튼을 y축을 기준으로 중앙에 배치하려고 할 때 9장의 요소 중앙 배치를 활용
- opacity:0.6으로 줘서 위로 올리기 전에는 약간 투명하게 함.

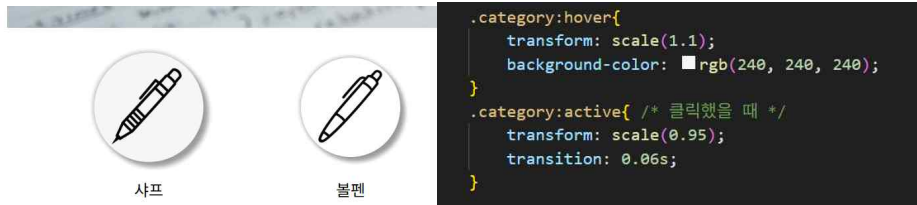
2.4. 날짜 및 시간 출력 기능

```
window.onload = function(){
  let years = document.getElementById("year_month_day_week");
  let time = document.getElementById("hour_minute_second");
  setInterval(function(){
    let now = new Date();
    let year = now.getFullYear();
    let month = now.getMonth() + 1;
    let day = now.getDate();
    let week = now.getDay(); // 0(일)~6(토)
    let hours = now.getHours();
    let min = now.getMinutes();
    let seconds = now.getSeconds();
    let weekDay = ['일요일', '월요일', '화요일', '수요일', '목요일', '금요일', '토요일'];

    years.innerHTML = year + "년 " + month + "월 " + day + "일 " + weekDay[week]; // 날짜
    if(min<10)
      min = "0" + min;
    if(seconds<10)
      seconds = "0" + seconds;
    if(hours >=0 && hours <12){
      if(hours==0)
        hours=12;
      hours = "AM &nbsp;"; + hours;
    }
    else{
      if(hours==12)
        hours = "PM &nbsp;"; + hours;
      else
        hours = "PM &nbsp;"; + (hours - 12);
    }
    time.innerHTML = hours + ":" + min + ":" + seconds;
  }, 500);
};
```

- Date()함수를 이용해서 날짜 출력
- 바로 사용하지 않고 요일, AM PM 같이 조금 더 가공해서 화면에 출력함
- day(일요일:0), month(1월:0)
- setInterval()을 500(0.5초)로 줬는데 1000으로 주게 된다면 다음 페이지를 이동할때마다 1초동안 기다려야 해서 조금 더 빨리 나타나게 함.

2.5. 그 외



- 이미지에 border-radius(6주차)를 적용해서 둥글게 만들고 hover와 active를 이용해서 모양이 바뀌게 함(이때 transform 속성은 회전하거나 확대/축소 및 모양, 크기, 위치 등을 자유롭게 바꾸는 속성이다. scale()은 태그를 확대, 축소하는 기능이다. transition은 해당 상태로 변할 때 즉시 변하지 않고 해당 초 동안 부드럽게 변하게 해준다)

```

// 제목 꾸미기
$(document).ready(function(){
  $('#big-title').slideDown(700); //슬라이드
  $('#small-title').delay(900).fadeIn(800);
});

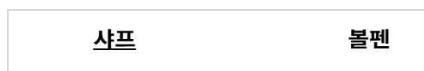
```

```

#content-title > #big-title{
  font-size: 100px;
  text-shadow: 4px 4px 8px #rgb(40,40,40);
  margin:20px 0;
  display:none; /*애니메이션을 적용하기 위해 처음에는
}
#content-title > #small-title{
  font-size: 30px;
  text-shadow: 4px 4px 8px #rgb(40,40,40);
  margin:20px 0;
  display:none;
}

```

- 슬라이더 중앙에 제목 부분을 그냥 출력하지 않고 약간의 애니메이션을 jquery를 이용해서 구현(13주차 강의자료 참고). 이때 나타나는 효과는 먼저 없어져 있어야 되기 때문에 display를 none으로 줌



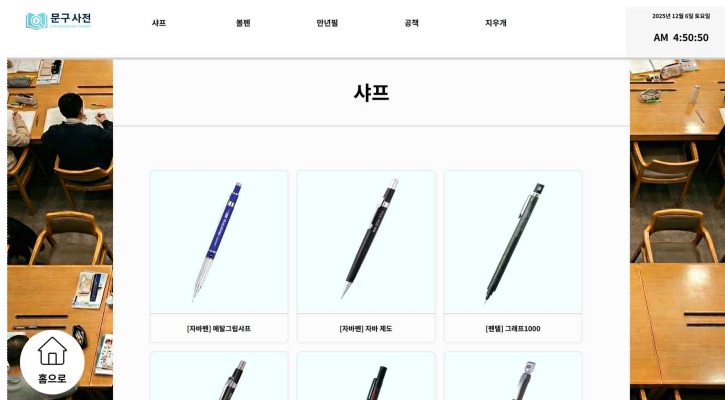
```

.tab> a:hover{
  text-decoration: underline;
}

```

- 상단 탭의 카테고리에도 hover를 적용
- 이때 css의 밑줄을 꾸미는 속성인 text-decoration를 이용했다.

3. 대분류 카테고리(category\~.html)



대분류 카테고리 중 하나

3.1. 홈으로 이동

- 대분류 카테고리부터는 홈(main.html)로 이동하는 수단을 제공함.



화면

```
#home{
  position:fixed;
  bottom:30px;
  text-align:center;
  left:30px;
  height:150px;
  width:150px;
  border-radius: 50%;
  overflow:hidden;
  background-color: white;
  color: black;
}
#home > h2{
  margin-top:-10px;
}
```

CSS

\category\homebutton\
homebuttonStyle.css

```
<a href= "../main.html">
  <div id="home">
    <img id="home-button" src= "../homebutton/home.png">
    <h2>홈으로</h2>
  </div>
</a>
```

HTML

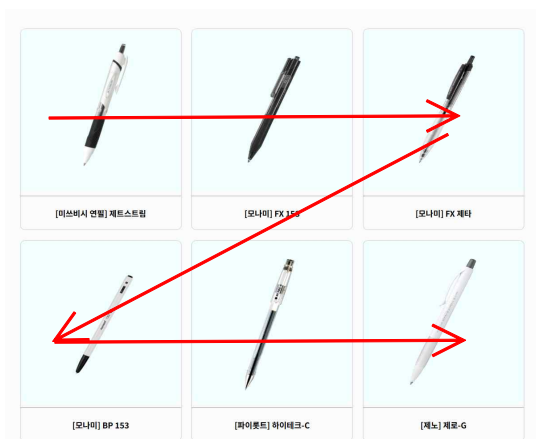
- 탭은 position이 fixed라서 계속 화면상에 보이는데 홈 버튼도 마찬가지다
- 이미지 아이콘과 h2 태그를 붙이고 링크를 main.html로 이동할 수 있도록 앵커(<a>)를 적용함.(이때 서브 카테고리和大분류 카테고리의 url이 차이가 있음)
- hover를 적용해서 마우스를 올리면 색이 변한다.
- 버튼 뿐만 아니라 로고를 클릭해도 홈으로 돌아가끔 <a>로 설정



시



3.2. 상품 목록 배치(\category\categorySource\productListStyle.css)



```
#text-body > ul{
  padding :75px;
  overflow:hidden;
}
.product-list{
  list-style:none;
  float:left;
  width:320px;
  height:400px;
  margin:10px;
  text-align:center;
  border-width: 0.5px;
  border-color: rgb(200, 200, 200);
  border-style:solid; /*태두리 종류가 있어
  border-radius: 10px;
  overflow:hidden;
}
```

- 9주차 강의 자료에서 나왔던 수평 정렬 레이아웃을 적용해서 상품의 리스트를 구성했다.(ul이 부모로 overflow를 적용했고 product-list는 float:left를 사용해 수평으로 정렬함)

- product-list의 overflow hidden은 상품 이미지의 div를 자식으로 가지고 있기



```
.product-background{
  display: inline-block;
  width:320px;
  border-radius:10px;
  background-color: ■rgb(235, 251, 255);
}
```

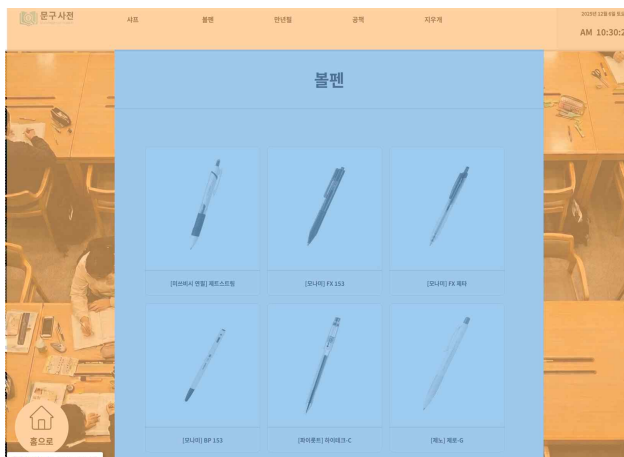
때문에 사용.

```
.product-image{
  transition:0.4s;
}
.product-image:hover{
  transform:translateY(-10px);
}
.product-background{
  display: inline-block;
  width:320px;
  border-radius:10px;
  background-color: ■rgb(235, 251, 255);
}
```

```
<li class="product-list"><div class="product-background"><a href="/ballpointPen/jetstream.html"><img class="product-image"
<li class="product-list"><div class="product-background"><a href="/ballpointPen/fx153.html"><img class="product-image" src=
<li class="product-list"><div class="product-background"><a href="/ballpointPen/fxzeta.html"><img class="product-image" src=
<li class="product-list"><div class="product-background"><a href="/ballpointPen/bp153.html"><img class="product-image" src=
<li class="product-list"><div class="product-background"><a href="/ballpointPen/hitec_c.html"><img class="product-image" sr
```

- 상품의 이미지에 배경색을 적용하게 된다면 hover의 transform 속성 때문에 마우스를 올리면 배경색까지 같이 움직여버림.(이때 translateY()는 해당 속성을 Y축으로 값만큼 이동시키는 함수다)
- product-background를 따로 만들어서 이미지와 배경이 따로 움직이도록 해서 좀더 자연스럽게 표현(마우스를 올리면 상품만 위로 살짝 이동됨)

3.3. 본문의 중앙 정렬 레이아웃(.categorySource\textBodyStyle.css)



```
#text-body{
  background: ■white;
  margin: 0 auto; /*중앙 정렬 레이아웃 9장 */
  margin-top: 120px;
  width:1200px;
  overflow:hidden;
  background-color: ■rgb(244, 244, 244);
}
```

- 9주차 자료의 중앙 정렬 레이아웃을 이용해서 전체적으로 내용이 중앙에 오도록 만들
- 서브 카테고리 내에서도 해당 본문 레이아웃을 그대로 이용.

3.4. 배경 이미지(.\background\backgroundStyle.css)

```
body{
  background-image: url('../background.png');
  background-size: cover; /*창 크기에 맞추면서도 일정 비율 유지*/
  background-position: center; /*배경 이미지의 배치 위치를 지정*/
  background-repeat: no-repeat;
  background-attachment: fixed;
}
```

- 이미지 반복은 사용하지 않고 완전 고정된 상태로 화면 전체를 꽉 채움.(6주차 강의 자료 참고함)
- background-size는 배웠던 100%를 사용하면 배경에 여백이 생기고 100% 100%를 사용하면 창크기에 따라 이미지가 찌그러져서 이상하게 보임
- cover이라는 지정한 요소를 다 덮도록 배경이미지를 확대하는 것으로 원본 이미지 비율을 유지. 화면 전체를 꽉 채우면서 창 크기를 줄여도 이미지 비율이 유지되는 효과를 줌.

4. css, 자바스크립트중 공통으로 많이 쓰는 형식은 외부 로 빼놓기.

homebutton

home.png

homebuttonStyle.css

tabClock

tabClock.js

tabDesign

mainTab.css

productSource

productMethod.js

productStyle.css

categorySource

productListStyle.css

textBodyStyle.css

홈버튼

시간

서브 카테고리(상품) 관련 공통 속성

대분류 카테고리의 본문 스타일, 상품 스타일

```
<link rel="stylesheet" href="../../tabDesign/mainTab.css">
<link rel="stylesheet" href="../../background/backgroundStyle.css"> <!-- 배경화면 디자인 -->
<link rel="stylesheet" href="../../categorySource/textBodyStyle.css"> <!-- 본문 내용 디자인 -->
<link rel="stylesheet" href="../../categorySource/productListStyle.css"> <!-- 제품 목록 디자인 -->
<link rel="stylesheet" href="../../homebutton/homebuttonStyle.css"> <!-- 홈 버튼 디자인 -->
<script src="../../tabClock/tabClock.js"> </script> <!-- 상단 시계 스크립트 -->
</head>
<body>
```

공통 스타일, 자바스크립트 적용 예시

- 구현 내용 특성상 공통된 스타일과 메소드를 사용하는 경우가 많음.
- 계속 같은 메소드, 스타일을 다른 html에서 복붙해서 사용하는건 가시성도 떨어지고 코드의 길이가 과도하게 길어지게 됨
- 2주차 강의 자료에서 외부 스타일, 외부에서 자바스크립트 내용을 부르는 방법의 내용을 참고함

- 모든 상품은 브랜드가 존재하고 각각 홈페이지가 존재함.
- 상품 설명에 있는 이미지 로고를 클릭하면 브랜드 홈페이지로 이동
- 가능하면 한국어 홈페이지로 연결했지만 존재하지 않는다면 영어나 일본어 홈페이지로 연결을 함.

5.2. 제품의 색 종류 표현

잉크 색상	(0.5, 0.7mm 한정) ●블랙 ●블루 ●레드 ●라이트블루 ●그린 ●라임그린 ●베이비핑크 ●퍼플
	(0.38, 1.0mm 한정) ●블랙 ●블루 ●레드

제조국	대한민국
색깔 종류	●블랙 ○화이트 ●블루 ●핑크

잉크 색상이나 제품 색을 더 직관적이게 표현

```
<tr>
  <th>잉크 색상</th>
  <td id="color">(0.5, 0.7mm 한정)<br>
    <div class="fig" style="background: <img alt="black square" style="width:10px; height:10px;"/> rgb(0, 0, 0)"></div><span>블랙</span> <!--style
    <div class="fig" style="background: <img alt="blue square" style="width:10px; height:10px;"/> rgb(0, 77, 161)"></div><span>블루</span>
    <div class="fig" style="background: <img alt="red square" style="width:10px; height:10px;"/> rgb(227, 0, 0)"></div><span>레드</span>
    <div class="fig" style="background: <img alt="light blue square" style="width:10px; height:10px;"/> rgb(138, 204, 242)"></div><span>라이트블루</span>
    <div class="fig" style="background: <img alt="green square" style="width:10px; height:10px;"/> rgb(0, 163, 105)"></div><span>그린</span>
  </td>
</tr>
```

HTML 부분

> notebook	44	
▼ productSource	45	.fig{
JS productMethod.js	46	display:inline-block;
# productStyle.css	47	width:18px;
<> ballpointPenCateg...	48	height:18px;
<> eraserCategory.html	49	margin:0px 2px;
	50	border-radius:50%;

css 부분(\category\productSource\productStyle.css)

- border-radius를 50%로 줘서 원형으로 표현하고 div속성을 span처럼 이용하기 위해서 display:inline-block를 사용함.
- 색은 캡처의 스포이드 기능으로 rgb 값을 추출함
- inline-block를 사용함으로써 텍스트 옆에 동그란 색 표현이 가능해짐.
- 색 명은 그냥 을 사용해서 구성.

5.3. 상세 설명 테이블

브랜드	 MITSUBISHI PENCIL 미쓰비시 연필
볼펜 종류	유성 볼펜
사이즈	11.3mm * 139.6mm (두께 * 길이)
무게	9.3g
심 종류	0.38mm, 0.5mm, 0.7mm, 1.0mm
제조국	일본
잉크 색상	(0.5, 0.7mm 한정) ●블랙 ●블루 ●레드 ●라이트블루 ●그린 ●라임그린 ●베이비핑크 ●퍼플
	(0.38, 1.0mm 한정) ●블랙 ●블루 ●레드

```
<div id="product-container">
  제트스트
  <table id="info">
    <tr>
      <th>브랜드</th>
      <td><a href="http://www
        <p>미쓰비시 연필</p>
    </tr>
    <tr>
      <th>볼펜 종류</th>
      <td>유성 볼펜</td>
    </tr>
  </table>
</div>
```

테이블 모양과 html 부분

```

}

#info{
  margin: 0 auto;
  width:700px;
  border:1.5px solid gray; /*두께, 선 종류, 색깔
  table-layout:fixed; /*테이블은 기본적으로 auto 속
  border-collapse:collapse; /*기본적으로 separate
}

#info th, #info td{
  border: 1.5px solid gray;
}

#info th{
  background-color: rgb(200, 200, 200);
  width:150px;
}

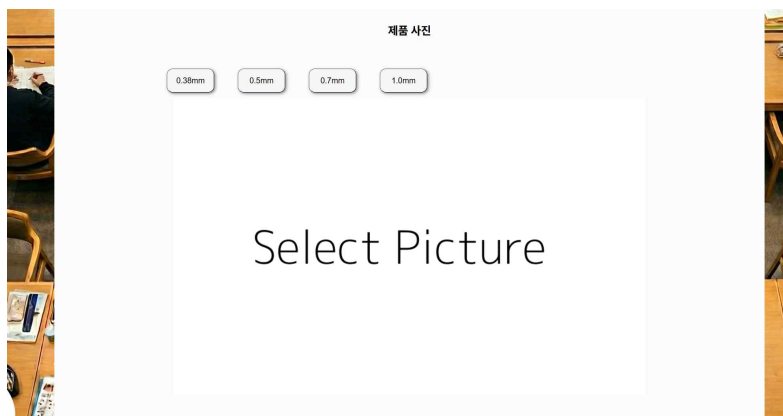
#info td{
  text-align:center;
  font-size:20px;
  font-weight:bold;
  color:rgb(0, 0, 123);
}

```

css 부분(\category\productSource\productStyle.css)

- <table>에 <table border="1">을 주면 테두리가 생기긴 하지만 원하는 모양을 표현하기에는 한계가 있음
- 직접 border 색상과 두께를 지정함
- 이때 **border-collapse**라는 속성이 있는데 반드시 **collapse**로 지정해야 함.
- border-collapse란 표의 테두리와 셀이 분리될지 말지를 결정하는 속성인데 디폴트 값으로 separate로 되어 있음. separate가 되면 셀과 겹 테두리가 따로 경계선을 가지게 되므로 collapse로 지정해서 셀과 표 테두리 부분이 따로 가지는 것을 막음.
- **table-layout:fixed**로 지정했는데 각 행의 길이를 원하는 만큼 지정하고 싶으면 필요하다. 기본 디폴트값이 auto로 지정되어 있는데 이 상태에서 width를 주게 된다면 width가 늘어난 상태에서 또 글의 내용에 따라 행의 길이가 달라지게 된다.

5.3. 상품의 다른 색상, 이미지를 보여주기



화면


```

<div id="select-product">
  <h3 align="center">제품 사진</h3><br>
  <input class="select-button" onclick="selectProduct(this, './image/jet0.38.png')" type="button" value="0.38mm"> <!--버튼-->
  <input class="select-button" onclick="selectProduct(this, './image/jet0.5.png')" type="button" value="0.5mm">
  <input class="select-button" onclick="selectProduct(this, './image/jet0.7.png')" type="button" value="0.7mm">
  <input class="select-button" onclick="selectProduct(this, './image/jet1.0.png')" type="button" value="1.0mm">
  <div id="selected-image">
    
    <!--이미지에 원하는 텍스트를 넣을 수 있다(한국어는 불가)-->
  </div>
</div>

```

HTML 부분

```

#select-product{
  margin: 50px auto;
  width: 900px;
  height: 700px;
  overflow: hidden;
}
.select-button{
  float: left;
  width: 80px;
  height: 40px;
  margin: 10px 0px 10px 40px;
  border-radius: 10px;
  border-width: 0.5px;
  box-shadow: 2px 2px 5px #gray;
  cursor: pointer; /*마우스를 올리면 손가락 모양으로 바뀐다 */
}
.selected{
  background-color: #rgb(150, 150, 150);
  color: #white;
}

```

```

.select-button:hover{
  background-color: #rgb(200, 200, 200);
}
.select-button:active{
  transform: scale(0.95);
}
#selected-image{
  margin: 10px auto;
  width: 800px;
  height: 500px;
  text-align: center;
  background-color: #white;
  overflow: hidden;
  clear: both;
}

```

CSS 부분(\category\productSource\productStyle.css)

- 12주차, 13주차 강의 자료에서 나왔던 dummyimage.com을 이용해서 이미지를 아직 선택하지 않을 때 보여주는 디폴트 이미지를 불러옴.
- dummyimage.com에 직접 가보면 배경 색과 원하는 텍스트를 입력하는 방법이 나온다.
- fff: 배경 색
- &text="": 원하는 텍스트 내용을 직접 입력(한국어는 입력하면 깨짐)
- **cursor** 속성은 해당 속성 위에 올라갈 때 마우스 커서의 모양을 정해주는 속성이다. pointer를 준다면 손가락 모양(☞)으로 바뀌게 만든다.
- 라디오 버튼을 이용하지 않고 <input type="button">을 이용해서 라디오 버튼처럼 동작하게끔 구현함.

5.3.1. 버튼 구현 방법(\category\productSource\productStyle.css)

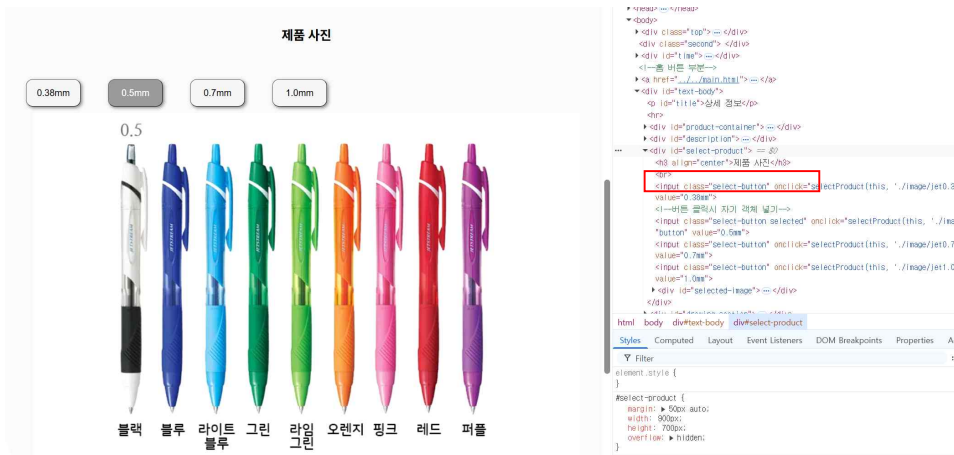
```

function selectProduct(btn , imageLink){
  let buttons = document.getElementsByClassName("select-button");
  for(let i=0; i<buttons.length; i++){
    buttons[i].classList.remove("selected"); //classList: 해당 요소
  }
  btn.classList.add("selected"); //선택된 버튼에 selected 클래스 추가
  document.getElementById("show").src=imageLink; //이미지 변경
}

```

- 'class='에는 여러 클래스가 들어올 수 있음. 따라서 버튼을 클릭했을 때 클릭한 버튼에 '선택된 버튼의 객체'를 전달해서 전체 버튼의 선택 클래스 해당 버튼에 선택되었다는 클래스(.selected)를 지우고 선택한 버튼에 해당 클래스를 삽입한다. 여기서 **classList**는 태그의 class를 쉽게 다룰 수 있도록 해주는 DOM 요소 객체다
- 버튼을 선택하면 메소드에 imageLink로 이미지 주소를 전달 받으므로 에서 src 부분을 매개변수로 받은 이미지 링크를 저장해서 이미지를 보여준다.

- .selected 클래스가 적용되면 해당 버튼이 선택된 것처럼 색이 변하게 된다.



선택된 이미지와 적용된 클래스

5.4. 캔버스 기능 구현하기

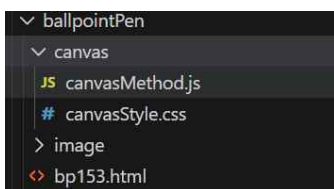


- 캔버스는 교재에 없는 내용이면서 수업 시간에 배운 내용이 아니기 때문에 외부에서 많이 자료를 가져와서 구현을 했다.

5.4.1 캔버스 기능의 메소드 및 자바스크립트

(category\ballpointPen\canvas\canvasMethod.js)

- 캔버스 기능은 잉크 색상이 있는 볼펜에서만 적용을 했다.



볼펜에서만 쓰기 때문에 볼펜 디렉토리 내에 외부 스타일, 자바스크립트로 구현

```
div id="color-palette">
  <span>볼펜 색상 선택: </span>
  <!-- onclick: 클릭하면 selectColor 함수 실행 -->
  <!-- this: 클릭한 버튼 자기 자신을 함수에 전달 -->
  <!-- 두 번째 인자: 해당 버튼의 색상값 전달 -->
  <button class="color-btn active" style="background-color: rgb(0, 0, 0);" onclick="selectColor(this, 'rgb(0, 0, 0)')"></button>
  <button class="color-btn" style="background-color: rgb(0, 56, 165);" onclick="selectColor(this, 'rgb(0, 56, 165)')"></button>
  <button class="color-btn" style="background-color: rgb(198, 12, 49);" onclick="selectColor(this, 'rgb(198, 12, 49)')"></button>
</div>
```

- 캔버스에 볼펜 잉크 색에 따라 적용시키기 위해서 버튼을 만들고 캔버스의 그리기 색을 지정한 색으로 변경하기 위한 메소드를 클릭 이벤트에 등록한다. 앞에서 상품 이미지를 보여줄 때 버튼을 라디오 버튼처럼 구현하는 것을 똑같이 사용하기 위해 자기 객체를 전달하고 지정한 rdb 색도 매개변수로 전달한다.

```
<!-- canvas: HTML5에서 그림을 그릴 수 있는 영역 -->
<!-- width, height: 캔버스 크기 설정 -->
<canvas id="drawing-canvas" width="800" height="400"
onmousedown="startDrawing(event)"onmousemove="draw(event)"onmouseup="stopDrawing()"onmouseout="stopDrawing()"></canvas>

<div id="canvas-delete">
  <button onclick="clearCanvas()">전체 지우기</button>
</div>
```

볼펜 상품 .html 아래에 있는 캔버스 태그

- 캔버스는 html 요소중 하나로 캔버스를 사용하기 위해서는 <canvas>로 태그를 써주면 된다.
- 캔버스를 쓰기 위해서 먼저 그릴 영역부터 잡아준다.
- 마우스로 그림을 그리기 때문에 각 마우스에 대한 이벤트를 canvas 태그에 함수를 등록한다.
- 여기서 등록한 메소드에 전달되는 **event 객체**는 이벤트 타입에 따라 발생하는 이벤트의 각종 정보가 들어 있는 객체 집합이다. 해당 객체에서 가장 필요한 정보는 마우스가 클릭한 위치에 대한 x, y 좌표로 캔버스에 그려지기 위해서는 반드시 마우스의 위치 정보가 필요하기 때문에 사용된다.
- 아래쪽에는 캔버스에 그린 것을 초기화 하기 위해 버튼을 하나 만들고 캔버스를 지우는 메소드를 등록한다.

```
let canvas, ctx; // canvas: 캔버스 요소, ctx: 캔버스에 그림을 그릴
let isDrawing = false; // 현재 그리기 중인지 (true: 그리기
let currentColor = 'rgb(0, 0, 0)'; // 현재 선택된 색상 (기본값

// window.onload: 페이지가 완전히 로드된 후 실행
window.addEventListener("load", function() { //load 이벤트 리스
  canvas = document.getElementById('drawing-canvas');
  ctx = canvas.getContext('2d');
});

// 그리기 시작 함수
function startDrawing(e) { // e: 마우스 이벤트 정보
  isDrawing = true; // 그리기 상태로 변경
  draw(e); // 클릭한 위치에 점 찍기 (바로 draw 함수 호출)
}
```

- 캔버스를 구현한 js 파일에 가보면 다음과 같은 모습을 볼 수 있다. 처음에는 변수를 초기화하고 이벤트 리스너를 등록하는 것을 볼 수 있다.

- 여기서 이벤트를 등록할 때 `window.addEventListener("load", {})`를 사용했는데 특정 이벤트가 발생했을 때 실행할 함수를 등록하는 기능을 제공한다. 사실 저 코드의 의미는 12주차 강의 자료에서 날짜를 출력했을 때 사용했던 `window.onload=function(){}`과 동일한 기능을 한다. 하지만 이걸 쓰지 않고 저 코드를 쓴 이유는 캔버스에도 `window.onload=function(){}`을 사용하게 된다면 로드가 되었을 때 `window.onload`가 캔버스 등록 메소드로 덮어쓰게 되어서 날짜에서도 사용되었던 `window.onload`가 더 이상 동작하지 않게 되기 때문이다. 따라서 캔버스가 동작하지만 시간을 더 이상 표시를 안 하게 되어서 다른 방법의 이벤트 등록 방법을 이용한 것이다.
- `startDrawing(e)` 메소드는 `onmousedown`(마우스를 눌렀을 때)에 등록한 함수로 누르고 있으면 그리기 모드가 동작하여 캔버스 위에 그려지게 된다.

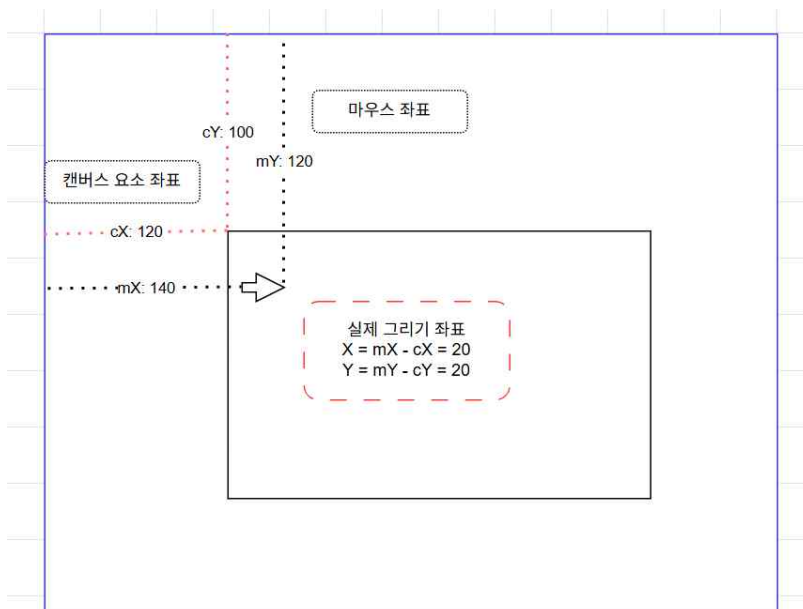
```
// 실제로 선을 그리는 함수
function draw(e) {
  if (!isDrawing) return; // 그리기 상태가 아니면 함수 종료

  // 캔버스 내에서의 마우스 좌표 계산
  const rect = canvas.getBoundingClientRect(); // 캔버스 크기
  const x = e.clientX - rect.left; // 마우스 x좌표 - 캔버스 왼쪽
  const y = e.clientY - rect.top; // 마우스 y좌표 - 캔버스 위쪽

  // 선 스타일 설정
  ctx.lineWidth = 3; // 선 굵기 3px
  ctx.lineCap = 'round'; // 선 끝 모양을 둥글게
  ctx.strokeStyle = currentColor; // 선 색상을 현재 선택한 색으로 설정

  // 선 그리기
  ctx.lineTo(x, y); // 현재 위치에서 (x, y)까지 선 경로 추가
  ctx.stroke(); // 경로를 따라 실제로 선 그리기
  ctx.beginPath(); // 새로운 경로 시작 (이전 경로와 분리)
  ctx.moveTo(x, y); // 펜을 (x, y) 위치로 이동
}
```

- 캔버스에 실제로 선을 그리는 메소드는 다음과 같다. 그리기 모드일때만 동작하게끔 했다
- 여기서 중요한 것은 마우스 좌표를 계산해서 마우스 위치에 그려지게 만드는 것이다. 여기에서 event 객체가 중요한 역할을 한다. 캔버스에 그려지려면 먼저 마우스의 좌표를 알아야 하는데 event 객체에서의 마우스 좌표는 창 의 왼쪽 끝(0,0)을 기준으로 좌표를 준다. 하지만 **캔버스의 그리기 좌표는 캔버스 내에서 왼쪽 끝을 기준으로 좌표를 계산한다.** 따라서 캔버스 내에서의 마우스 그리기 좌표를 계산하기 위해서는 마우스 좌표에서 캔버스 요소의 창 내에서의 위치 좌표값을 빼야 정확한 그리기 좌표가 나오게 된다.
`canvas.getBoundingClientRect()` 함수를 이용하게 된다면 요소의 크기와 화면 상의 위치를 리턴하게 되고 그 값을 이용해서 그리기 좌표를 계산한다.



그리기 좌표 계산식

- 좌표가 준비가 되었으면 선 굵기, 모양, 색을 지정하고 실제로 그리는 것을 시작한다. 메소드의 기능은 다음과 같다.
 - `lineTo()`: 지금까지의 경로(path)에서 마지막 점에서 (x, y) 까지 선을 그음
 - `stroke()`: 만들어진 경로에 실제로 선을 그림
 - `beginPath()`: 새로운 경로를 다시 만들. 연필을 떼서 다시 시작한다는 것으로 직선이 아니라 자유롭게 그릴 수 있게 만들.
 - `moveTo()`: 시작점을 지정, 시작점을 지정하고 다시 호출하면 `lineTo`가 호출되어 이어진 모습으로 그려지게 됨.

```
// 그리기 중지 함수
function stopDrawing() {
  isDrawing = false; // 그리기 상태 해제
  ctx.beginPath(); // 경로 초기화 (다음 그리기를 위해)
}

// 색상 선택 함수
function selectColor(btn, color) { // btn: 클릭한 버튼, color: 선택한 색상
  // 모든 색상 버튼에서 active 클래스 제거
  btns = document.getElementsByClassName('color-btn');
  for(let i=0; i<btns.length; i++){
    btns[i].classList.remove('active');
  }
  btn.classList.add('active'); // 클릭한 버튼에만 active 클래스 추가
  currentColor = color; // 현재 색상을 클릭한 버튼의 색상으로 설정
}

// 캔버스 전체 지우기 함수
function clearCanvas() {
  ctx.clearRect(0, 0, canvas.width, canvas.height); // (0, 0)에서 (canvas.width, canvas.height)까지 지움
}
```

- 그리기 중지 함수는 마우스 버튼을 놓을 때 동작하는 함수로 그리기 모드를 중지시키고 경로를 완전히 초기화 시킨다. 이렇게 하면 새로 선을 그릴 때 이전에 그린 선과 직선으로 이어지게 되는 현상을 막는다

- selectColor()는 색상을 선택하는 버튼을 라디오 버튼처럼 구현한 것으로 앞에서 봤던 상품 이미지 선택 버튼과 동작 방식이 비슷하다. 대신 캔버스의 색상을 지정하기 위해서 currentColor 변수에 매개변수로 받은 색상을 저장한다.
- clearCanvas()는 캔버스 전체를 초기화 해주는 함수로 전체 삭제 버튼을 클릭하면 실행이 된다. clearRect()는 0부터 캔버스 높이와 길이만큼 사각형 영역으로 지우는 메소드다.

5.4.2. CSS 속성(category\ballpointPen\canvas\canvasStyle.css)

```
.color-btn {
  width: 35px;
  height: 35px;
  border-radius: 50%; /* 원형 버튼으로 만들기 */
  border: 2px solid #000080; /* rgb(0, 0, 128) */
  margin: 5px;
  cursor: pointer; /* 마우스 올리면 손가락 모양 */
  transition: 0.2s; /* 크기 변화 애니메이션 0.2초 */
}

.color-btn:hover {
  transform: scale(1.1); /* 마우스 올리면 1.1배 커짐 */
}

.color-btn.active {
  border: 3px solid #000080; /* 선택된 버튼은 테두리 굵게 */
  box-shadow: 0 0 10px #000080; /* 선택된 버튼에 그림자 효과 */
}

#drawing-canvas {
  border: 2px solid #000080;
  background-color: white;
  cursor: crosshair; /* 캔버스 위에서 십자 모양 커서 */
}

#canvas-controls {
  margin-top: 15px;
}
```

- 캔버스의 CSS 속성은 특별한 건 없다. 지금까지 해온것처럼 hover를 통해 마우스가 위로 올라오면 다른 모습으로 바뀌고 배경색, 위치 조정이 대부분이다.
- 그래도 새로운 것을 말하자면 cursor:crosshair가 있다. 이걸 적용하면 해당 요소(캔버스) 위에 마우스가 올라가게 된다면 마우스 커서가 십자가(+) 모양으로 바뀌게 된다.



전체 화면 모습과 선택된 버튼 클래스

5.5. 뒤로 가기 버튼



- 맨 아래쪽에는 뒤로 가기 버튼을 만들어서 다시 해당 상품에 속한 대분류 카테고리로 이동한다.

```
</div>  
  
<input type="button" id="back-button" value="뒤로 가기" onclick="location.href='../ballpointPenCategory.html'">  
</div>
```

- 방법은 버튼을 클릭하면 이전 대분류 카테고리로 링크를 바꾸는 방법을 이용했다.

6. 마무리

지금까지 구현한 홈페이지에 대해서 설명을 했다. 이 프로젝트를 하면서 느낀점은 디자인이라는 것은 생각보다 많은 시간이 든다는 것을 알게 되었고 속성을 여러 가지 적용해보면 생각되던 적용이 안되는 것이 많다는 것을 알게 되었다. 문구에 대한 내용은 원래 색상보다 더 많은 색을 가지고 있고 누락된 내용이 있지만 홈페이지 내용의 정확정보다는 필요한 기능부터 구현하는 것이 중요하다고 생각해서 일부 내용을 제외했다. 원래 목표하던 이미지 슬라이드 기능을 마무리 할 수 있어서 좋았고 이번 프로젝트를 통해 프론트엔드의 html, css, javascript, jquery 에 대해서 단순히 개념만 아는 것보다 많이 배우게 시간이 된것같아서 좋았다. 다음에 시간이 있다면 해당 홈페이지를 조금더 이쁘게 구현하고 싶다.